UATP Capsule Engine System Guide

Table of Contents

1. [Executive Summary](#executive-summary)
2. [System Architecture](#system-architecture)
3. [Core Components](#core-components)
4. [API Reference](#api-reference)
5. [Federation System](#federation-system)
6. [Payment Integration](#payment-integration)
7. [Rights Management](#rights-management)
8. [Compliance Framework](#compliance-framework)
9. [Development Guide](#development-guide)
10. [Deployment Guide](#deployment-guide)
11. [Operations & Monitoring](#operations--monitoring)
12. [Security](#security)
13. [Troubleshooting](#troubleshooting)

Executive Summary

The **UATP Capsule Engine** is a comprehensive AI reasoning trace management sys

Key Features

- **Federated AI Model Registry**: Multi-organizational AI model collaboration w
- **Reasoning Trace Capture**: Comprehensive AI reasoning process documentation
- **Payment Integration**: Multi-provider payment processing with intelligent ro
- **Rights Management**: Intellectual property tracking and licensing automation
- **Compliance Framework**: Multi-regulatory compliance (GDPR, AI Act, NIST AI R
- **RESTful API**: Complete HTTP API for system integration
- **Real-time Monitoring**: Comprehensive observability and health monitoring

Target Use Cases

- Enterprise AI model management and governance
- Multi-organizational AI collaboration platforms
- AI reasoning audit and compliance systems
- Intellectual property licensing for AI models
- Federated AI research networks

System Architecture

High-Level Architecture

```
……
·                    Client Applications                    ·
```

```
┌─────────────────────────────────────────────────────────────┐
│                        API Gateway                          │
│                     (Quart Framework)                       │
├─────────────────────────────────────────────────────────────┤
│  Core Engine  │  Federation  │  Payments  │  Rights  │ Compliance │
│   Services    │   Registry   │   System   │   Mgmt   │ Framework  │
├─────────────────────────────────────────────────────────────┤
│                    Data Persistence Layer                   │
│            (SQLite/PostgreSQL, File Storage)                │
├─────────────────────────────────────────────────────────────┤
│              External Integrations & Services               │
│    (Stripe, PayPal, Crypto, Federation Members)             │
└─────────────────────────────────────────────────────────────┘
```

Core Architectural Principles

- **Modular Design**: Loosely coupled components with clear interfaces
- **Federated Architecture**: Distributed trust and validation across organizati
- **Event-Driven**: Asynchronous processing with event-based communication
- **Scalable**: Horizontal scaling support for high-volume deployments
- **Secure**: End-to-end encryption and comprehensive audit trails

---

Core Components

1. Capsule Engine (`src/engine/capsule_engine.py`)

The heart of the system, managing AI reasoning trace capsules.

Key Features:
- Capsule creation, sealing, and verification
- Metadata management and indexing
- Status tracking and lifecycle management
- Integration with external validation services

Core Methods:
```python
# Create and seal a reasoning trace capsule
capsule = await engine.create_capsule(
    capsule_type="reasoning_trace",
    content=reasoning_data,
    metadata={"model": "gpt-4", "task": "analysis"}
)

# Retrieve capsule with verification
capsule = await engine.get_capsule(capsule_id)

# Update capsule status
```

```
await engine.update_capsule_status(capsule_id, CapsuleStatus.VERIFIED)
```

2. Federated Registry (`src/integrations/federated_registry.py`)

Manages multi-organizational AI model collaboration and validation.

Key Features:
- Federation member management with role-based access
- Trust-based validation across organizations
- Distributed capsule verification
- Cross-organizational trace aggregation

Federation Roles:
- **COORDINATOR**: Manages registry synchronization
- **VALIDATOR**: Validates capsules and traces
- **MEMBER**: Regular federation participant
- **OBSERVER**: Read-only access to federation data

Core Operations:
```python
# Register model with federation
capsule_id = await registry.register_model_with_federation(
    model_id="gpt-4-advanced",
    provider="openai",
    access_level="restricted"
)

# Verify capsule across federation
verification = await registry.verify_federated_capsule(
    capsule, required_validators=3
)

# Aggregate traces from federation members
traces = await registry.aggregate_federated_traces({
    "provider": "openai",
    "after": "2024-01-01T00:00:00Z"
})
```

3. Payment Integration (`src/payments/`, `src/services/`)

Comprehensive payment processing with multiple provider support.

Supported Providers:
- **Stripe**: Credit/debit cards, ACH transfers
- **PayPal**: PayPal balance, linked accounts
- **Cryptocurrency**: Bitcoin, Ethereum, stablecoins
- **Bank Transfers**: Direct bank account transfers

Payment Workflows:
- **Payouts**: User earnings distribution
- **Refunds**: Transaction reversals
- **License Fees**: IP licensing payments
- **Dividend Payments**: Revenue sharing

Intelligent Routing:
- Lowest fee optimization
- Fastest processing
- Highest reliability
- Load balancing across providers

4. Rights Management (`src/rights/`)

Intellectual property tracking and licensing automation.

Rights Evolution System:
- **Creation**: Initial IP registration
- **Assignment**: Rights transfer between parties
- **Licensing**: Usage rights distribution
- **Enforcement**: Automated compliance monitoring

Bond Citizenship System:
- **Citizenship Bonds**: Stakeholder participation tokens
- **Governance Rights**: Voting and decision-making privileges
- **Revenue Sharing**: Automated dividend distribution

5. Compliance Framework (`src/compliance/`)

Multi-regulatory compliance management and monitoring.

Supported Frameworks:
- **GDPR**: European data protection compliance
- **AI Act**: European AI regulation compliance
- **NIST AI RMF**: US AI risk management framework

Compliance Operations:
- Automated compliance assessments
- Risk scoring and mitigation planning
- Audit trail generation
- Regulatory reporting

API Reference

The system provides a comprehensive RESTful API with the following endpoints:

Authentication

All API endpoints require API key authentication:

```http
Authorization: Bearer your-api-key-here
```

Core Engine API (`/api/v1/engine/`)

Create Capsule

```http
POST /api/v1/engine/capsules
Content-Type: application/json

{
  "capsule_type": "reasoning_trace",
  "content": "base64-encoded-reasoning-data",
  "metadata": {
    "model": "gpt-4",
    "task": "analysis",
    "timestamp": "2024-01-01T00:00:00Z"
  }
}
```

Get Capsule

```http
GET /api/v1/engine/capsules/{capsule_id}
```

Update Capsule Status

```http
PUT /api/v1/engine/capsules/{capsule_id}/status
Content-Type: application/json

{
  "status": "verified",
  "verification_details": {
    "validator": "system",
    "timestamp": "2024-01-01T00:00:00Z"
  }
}
```

Federation API (`/api/v1/federation/`)

Register Model

```http
POST /api/v1/federation/models
Content-Type: application/json
```

```
{
  "model_id": "gpt-4-advanced",
  "provider": "openai",
  "access_level": "restricted",
  "metadata": {
    "capabilities": ["text", "code"],
    "version": "1.0"
  }
}
```

Verify Capsule

```http
POST /api/v1/federation/capsules/{capsule_id}/verify
Content-Type: application/json

{
  "required_validators": 3
}
```

Query Federated Traces

```http
GET /api/v1/federation/traces?provider=openai&after=2024-01-01T00:00:00Z
```

Payment API (`/api/v1/payments/`)

Initiate Payout

```http
POST /api/v1/payments/payouts
Content-Type: application/json

{
  "user_id": "user_123",
  "amount": 50.00,
  "currency": "USD",
  "description": "Monthly earnings payout"
}
```

Configure Payout Method

```http
POST /api/v1/payments/payout-methods
Content-Type: application/json

{
  "user_id": "user_123",
```

```
  "payout_method": "paypal",
  "payout_details": {
    "email": "user@example.com"
  }
}
```

Process Bulk Payouts

```http
POST /api/v1/payments/payouts/bulk
Content-Type: application/json

{
  "payouts": [
    {"user_id": "user_1", "amount": 25.00},
    {"user_id": "user_2", "amount": 75.00}
  ],
  "batch_description": "Weekly contributor payouts"
}
```

Rights Management API (`/api/v1/rights/`)

Create Rights Evolution

```http
POST /api/v1/rights/evolution
Content-Type: application/json

{
  "asset_id": "model_xyz",
  "evolution_type": "license_grant",
  "party_from": "creator_id",
  "party_to": "licensee_id",
  "terms": {
    "license_type": "commercial",
    "duration": "1_year",
    "territory": "global"
  }
}
```

Issue Citizenship Bond

```http
POST /api/v1/rights/bonds
Content-Type: application/json

{
  "citizen_id": "citizen_123",
```

```
  "bond_type": "governance",
  "value": 1000.00,
  "privileges": ["voting", "revenue_sharing"]
}
```

Compliance API (`/api/v1/compliance/`)

Conduct Assessment

```http
POST /api/v1/compliance/assessments
Content-Type: application/json

{
  "framework": "gdpr",
  "system_id": "production_system",
  "assessor_id": "compliance_officer",
  "evidence": {
    "data_processing_activities": ["user_data", "model_training"],
    "privacy_measures": ["encryption", "anonymization"]
  }
}
```

Generate Compliance Report

```http
POST /api/v1/compliance/reports
Content-Type: application/json

{
  "reporting_period_days": 90,
  "frameworks": ["gdpr", "ai_act"],
  "include_trends": true
}
```

Federation System

Federation Architecture

The federation system enables multiple organizations to collaborate on AI model

Federation Members

Each member has:
- **Unique ID**: Cryptographic identity
- **Role**: COORDINATOR, VALIDATOR, MEMBER, or OBSERVER

- **Trust Score**: 0.0-1.0 reliability rating
- **Endpoint**: API endpoint for federation communication
- **Verification Key**: Public key for cryptographic verification

Trust-Based Validation

The system uses a distributed trust model:
1. **Trust Threshold**: Minimum score (default 0.7) for member participation
2. **Required Validators**: Minimum number of validators for verification
3. **Weighted Voting**: Trust scores influence validation decisions
4. **Circuit Breaker**: Automatic member isolation on repeated failures

Federation Workflows

Model Registration:
1. Member registers model locally
2. Creates federation capsule with model metadata
3. Distributes capsule to federation members
4. Members validate and store capsule
5. Model becomes available across federation

Trace Aggregation:
1. Query sent to all trusted federation members
2. Members respond with matching traces
3. Results aggregated and deduplicated
4. Trust scores applied to weight results
5. Consolidated response returned

Governance Decisions:
1. Member proposes policy change
2. Proposal distributed to federation
3. Members vote based on trust weights
4. Decision applied if consensus reached
5. All members notified of changes

Federation Security

- **Cryptographic Signatures**: All federation communications signed
- **Mutual Authentication**: TLS with client certificates
- **Audit Trails**: Complete log of federation activities
- **Rate Limiting**: Protection against abuse and DoS attacks

---

Payment Integration

Payment System Architecture

The payment system provides enterprise-grade payment processing with multiple pr

Payment Processor Manager

Features:
- Multi-provider support (Stripe, PayPal, Crypto)
- Intelligent routing based on:
  - Lowest fees
  - Fastest processing
  - Highest reliability
  - Load balancing
- Circuit breaker pattern for fault tolerance
- Real-time health monitoring

Routing Strategies:
```python
# Lowest fee routing
result = await processor.process_payment(
    amount=Decimal("100.00"),
    currency="USD",
    recipient_details=recipient,
    routing_strategy=RoutingStrategy.LOWEST_FEE
)

# Fastest processing
result = await processor.process_payment(
    amount=Decimal("100.00"),
    currency="USD",
    recipient_details=recipient,
    routing_strategy=RoutingStrategy.FASTEST
)
```

Payment Workflows

Payout Workflow:
1. **User Validation**: Verify user identity and balance
2. **Payout Method Check**: Ensure valid payment method configured
3. **Fee Calculation**: Calculate processing fees across providers
4. **Payment Routing**: Select optimal payment processor
5. **Payment Processing**: Execute payment through selected provider
6. **Confirmation**: Verify payment completion
7. **Balance Update**: Update user account balances
8. **Notifications**: Send completion notifications

Refund Workflow:
1. **Refund Validation**: Verify refund eligibility and amount
2. **Original Transaction Lookup**: Find original payment details
3. **Refund Calculation**: Calculate refund amount including fees
4. **Refund Processing**: Execute refund through original processor

5. **Record Update**: Update transaction records
6. **Confirmation**: Send refund confirmation

Payment Methods

PayPal Integration:
- PayPal balance transfers
- Linked bank account payments
- International currency support
- Webhook event processing

Stripe Integration:
- Credit/debit card payments
- ACH bank transfers
- International wire transfers
- Real-time payment status updates

Cryptocurrency Integration:
- Bitcoin payments
- Ethereum payments
- Stablecoin support (USDC, USDT)
- Blockchain confirmation tracking

Payment Security

- **PCI DSS Compliance**: Credit card data protection
- **Encryption**: End-to-end payment data encryption
- **Fraud Detection**: Real-time fraud monitoring
- **Audit Logging**: Comprehensive payment audit trails
- **Webhook Verification**: Cryptographic webhook signature validation

---

Rights Management

Rights Evolution System

The rights evolution system tracks intellectual property rights throughout their

Rights Evolution Types

Creation Rights:
- Initial IP registration
- Ownership establishment
- Rights documentation
- Metadata attribution

Assignment Rights:
- Ownership transfer
- Rights conveyance

- Legal documentation
- Consideration tracking

Licensing Rights:
- Usage rights grants
- License term management
- Royalty calculations
- Compliance monitoring

Enforcement Rights:
- Violation detection
- Automated enforcement
- Legal action initiation
- Damage calculations

Rights Evolution Workflow

```python
# Create initial rights
evolution = await rights_engine.create_evolution(
    asset_id="ai_model_123",
    evolution_type=RightsEvolutionType.CREATION,
    party_from=None,
    party_to="creator_id",
    terms={
        "creation_date": "2024-01-01",
        "rights_type": "intellectual_property",
        "scope": "global"
    }
)

# License rights to third party
license_evolution = await rights_engine.create_evolution(
    asset_id="ai_model_123",
    evolution_type=RightsEvolutionType.LICENSE_GRANT,
    party_from="creator_id",
    party_to="licensee_id",
    terms={
        "license_type": "commercial",
        "duration": "1_year",
        "territory": "north_america",
        "royalty_rate": 0.05
    }
)
```

Bond Citizenship System

The bond citizenship system enables stakeholder participation in the ecosystem t

Bond Types

Governance Bonds:
- Voting rights in system decisions
- Policy change participation
- Fee structure decisions
- Strategic direction input

Revenue Bonds:
- Dividend distribution rights
- Fee sharing participation
- Performance-based rewards
- Long-term value capture

Utility Bonds:
- Service access rights
- Priority processing
- Enhanced features
- Technical support

Bond Management

```python
# Issue governance bond
bond = await citizenship_engine.issue_bond(
    citizen_id="stakeholder_123",
    bond_type=BondType.GOVERNANCE,
    value=Decimal("1000.00"),
    privileges=["voting", "policy_input"],
    terms={
        "voting_weight": 0.1,
        "term_length": "perpetual",
        "transfer_restrictions": "none"
    }
)

# Distribute dividends
dividend_result = await citizenship_engine.distribute_dividends(
    total_amount=Decimal("10000.00"),
    distribution_criteria="proportional_holdings"
)
```

Rights Enforcement

Automated Monitoring:
- Usage pattern analysis

- License term compliance
- Violation detection
- Anomaly identification

Enforcement Actions:
- Automated warnings
- License suspension
- Legal notice generation
- Damage calculation

Dispute Resolution:
- Mediation system
- Arbitration support
- Evidence collection
- Resolution tracking

Compliance Framework

Regulatory Compliance Architecture

The compliance framework provides comprehensive regulatory compliance management

Supported Frameworks

GDPR (General Data Protection Regulation):
- Data processing lawfulness
- Privacy rights implementation
- Data breach notification
- Privacy impact assessments

AI Act (European AI Regulation):
- AI system classification
- Risk assessment procedures
- Conformity assessment
- CE marking requirements

NIST AI RMF (Risk Management Framework):
- AI risk identification
- Risk mitigation strategies
- Continuous monitoring
- Incident response procedures

Compliance Assessment Process

```python
# Conduct GDPR assessment
assessment = await compliance_framework.conduct_compliance_assessment(
    framework=RegulatoryFramework.GDPR,
```

```
    system_id="production_system",
    assessor_id="compliance_officer",
    evidence={
        "data_processing_activities": ["user_profiles", "model_training"],
        "privacy_measures": ["encryption", "anonymization", "access_controls"],
        "data_retention_policies": ["automated_deletion", "user_requests"],
        "breach_procedures": ["incident_response", "notification_system"]
    }
)
```

Generate compliance report

```
report = await compliance_framework.generate_compliance_report(
    reporting_period_days=90,
    include_frameworks=[RegulatoryFramework.GDPR, RegulatoryFramework.AI_ACT]
)
```

Compliance Monitoring

Continuous Assessment:
- Real-time compliance monitoring
- Automated risk detection
- Policy violation alerts
- Trend analysis

Reporting:
- Regulatory report generation
- Audit trail maintenance
- Compliance dashboard
- Executive summaries

Risk Management:
- Risk scoring algorithms
- Mitigation plan generation
- Remediation tracking
- Escalation procedures

Development Guide

Prerequisites

- Python 3.9+
- pip package manager
- Git version control
- SQLite (development) or PostgreSQL (production)

Installation

1. **Clone Repository:**
```bash
git clone https://github.com/your-org/uatp-capsule-engine.git
cd uatp-capsule-engine
```

2. **Create Virtual Environment:**
```bash
python -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate
```

3. **Install Dependencies:**
```bash
pip install -r requirements.txt
```

4. **Configure Environment:**
```bash
cp .env.example .env
# Edit .env with your configuration
```

Configuration

Environment Variables:
```bash
# Database
DATABASE_URL=sqlite:///capsule_engine.db

# API Keys
STRIPE_API_KEY=sk_test_your_stripe_key
PAYPAL_CLIENT_ID=your_paypal_client_id
PAYPAL_CLIENT_SECRET=your_paypal_secret

# Federation
UATP_SIGNING_KEY=your_ed25519_private_key
FEDERATION_ENDPOINT=https://your-federation-endpoint.com

# Security
JWT_SECRET_KEY=your_jwt_secret
API_KEY_SALT=your_api_key_salt
```

Running the Application

Development Mode:
```bash
```

```
python src/main.py
```

Production Mode:
```bash
gunicorn -w 4 -k uvicorn.workers.UvicornWorker src.main:app
```

Testing

Run All Tests:
```bash
python -m pytest tests/ -v
```

Run Specific Test Category:
```bash
python -m pytest tests/test_engine/ -v
python -m pytest tests/test_federation/ -v
python -m pytest tests/test_payments/ -v
```

Test Coverage:
```bash
python -m pytest --cov=src tests/
```

Code Style

The project uses:
- **Black**: Code formatting
- **isort**: Import sorting
- **flake8**: Linting
- **mypy**: Type checking

Format Code:
```bash
black src/ tests/
isort src/ tests/
```

Check Code Quality:
```bash
flake8 src/ tests/
mypy src/
```

Deployment Guide

Docker Deployment

Build Docker Image:
```bash
docker build -t uatp-capsule-engine .
```

Run Container:
```bash
docker run -d \
  --name capsule-engine \
  -p 8000:8000 \
  -e DATABASE_URL=postgresql://user:pass@db:5432/capsule_engine \
  -e STRIPE_API_KEY=sk_live_your_stripe_key \
  uatp-capsule-engine
```

Docker Compose:
```yaml
version: '3.8'

services:
  app:
    build: .
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgresql://postgres:password@db:5432/capsule_engine
      - STRIPE_API_KEY=sk_live_your_stripe_key
    depends_on:
      - db
      - redis

  db:
    image: postgres:13
    environment:
      - POSTGRES_DB=capsule_engine
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=password
    volumes:
      - postgres_data:/var/lib/postgresql/data

  redis:
    image: redis:6-alpine
    ports:
      - "6379:6379"

volumes:
  postgres_data:
```

```

## Kubernetes Deployment

**Deployment Configuration:**
```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: capsule-engine
spec:
  replicas: 3
  selector:
    matchLabels:
      app: capsule-engine
  template:
    metadata:
      labels:
        app: capsule-engine
    spec:
      containers:
      - name: capsule-engine
        image: uatp-capsule-engine:latest
        ports:
        - containerPort: 8000
        env:
        - name: DATABASE_URL
          valueFrom:
            secretKeyRef:
              name: capsule-engine-secrets
              key: database-url
        - name: STRIPE_API_KEY
          valueFrom:
            secretKeyRef:
              name: capsule-engine-secrets
              key: stripe-api-key
        resources:
          requests:
            memory: "256Mi"
            cpu: "250m"
          limits:
            memory: "512Mi"
            cpu: "500m"
```

Service Configuration:
```yaml
apiVersion: v1
```
```

```
kind: Service
metadata:
  name: capsule-engine-service
spec:
  selector:
    app: capsule-engine
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8000
  type: LoadBalancer
```

Production Considerations

Database:
- Use PostgreSQL for production
- Enable connection pooling
- Configure regular backups
- Set up read replicas for scaling

Security:
- Use HTTPS with valid SSL certificates
- Enable CORS protection
- Configure rate limiting
- Set up Web Application Firewall (WAF)

Monitoring:
- Deploy Prometheus for metrics collection
- Set up Grafana for visualization
- Configure alerting for critical issues
- Enable distributed tracing

Scaling:
- Use horizontal pod autoscaling
- Configure load balancing
- Set up database sharding if needed
- Implement caching strategies

Operations & Monitoring

Health Monitoring

System Health Endpoints:
```http
GET /api/v1/health
GET /api/v1/engine/health
```

```
GET /api/v1/federation/health
GET /api/v1/payments/health
GET /api/v1/compliance/health
```

Health Check Response:
```json
{
  "status": "healthy",
  "timestamp": "2024-01-01T00:00:00Z",
  "version": "1.0.0",
  "components": {
    "database": "healthy",
    "redis": "healthy",
    "external_apis": "healthy"
  },
  "metrics": {
    "active_capsules": 1250,
    "federation_members": 5,
    "pending_payments": 23,
    "compliance_assessments": 45
  }
}
```

Metrics Collection

Prometheus Metrics:
- Request rates and latencies
- Error rates by endpoint
- Database connection pool usage
- Payment processing success rates
- Federation member health status

Custom Metrics:
```python
# Capsule metrics
capsule_creation_total = Counter('capsule_creation_total', 'Total capsules creat
capsule_verification_duration = Histogram('capsule_verification_duration_seconds

# Federation metrics
federation_request_total = Counter('federation_request_total', 'Total federation
federation_member_health = Gauge('federation_member_health', 'Federation member

# Payment metrics
payment_processing_total = Counter('payment_processing_total', 'Total payments p
payment_success_rate = Gauge('payment_success_rate', 'Payment success rate')
```

Logging

Log Configuration:

```python
LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'formatters': {
        'standard': {
            'format': '%(asctime)s [%(levelname)s] %(name)s: %(message)s'
        },
        'json': {
            'format': '{"timestamp": "%(asctime)s", "level": "%(levelname)s", "l
        }
    },
    'handlers': {
        'console': {
            'class': 'logging.StreamHandler',
            'formatter': 'standard',
            'level': 'INFO'
        },
        'file': {
            'class': 'logging.handlers.RotatingFileHandler',
            'filename': 'capsule_engine.log',
            'maxBytes': 10485760,  # 10MB
            'backupCount': 5,
            'formatter': 'json',
            'level': 'DEBUG'
        }
    },
    'loggers': {
        'capsule_engine': {
            'handlers': ['console', 'file'],
            'level': 'DEBUG',
            'propagate': False
        }
    }
}
```

Alerting

Critical Alerts:

- Database connection failures
- Payment processor outages
- Federation member disconnections
- Compliance assessment failures

- Security breach attempts

Alert Configuration:
```yaml
# Prometheus AlertManager
groups:
- name: capsule_engine
  rules:
  - alert: DatabaseDown
    expr: up{job="capsule-engine-db"} == 0
    for: 30s
    labels:
      severity: critical
    annotations:
      summary: "Database is down"
      description: "Database has been down for more than 30 seconds"

  - alert: PaymentProcessorError
    expr: payment_processor_error_rate > 0.1
    for: 5m
    labels:
      severity: warning
    annotations:
      summary: "High payment processor error rate"
      description: "Payment processor error rate is above 10%"
```

Backup and Recovery

Database Backup:
```bash
# Daily backup
pg_dump capsule_engine > backup_$(date +%Y%m%d).sql

# Restore from backup
psql capsule_engine < backup_20240101.sql
```

File Storage Backup:
```bash
# Backup capsule files
tar -czf capsule_files_$(date +%Y%m%d).tar.gz /var/lib/capsule_engine/files/

# Restore files
tar -xzf capsule_files_20240101.tar.gz -C /var/lib/capsule_engine/
```

Security

Authentication & Authorization

API Key Management:
- Scoped API keys (read, write, admin)
- Key rotation policies
- Usage monitoring and rate limiting
- Audit trail for key usage

JWT Token Security:
- Short-lived access tokens
- Secure refresh token storage
- Token blacklisting support
- Cryptographic signature verification

Data Protection

Encryption:
- Data at rest: AES-256 encryption
- Data in transit: TLS 1.3
- API communications: HTTPS only
- Database: Transparent Data Encryption (TDE)

Personal Data Protection:
- GDPR compliance measures
- Data anonymization techniques
- Right to be forgotten implementation
- Consent management system

Network Security

API Security:
- Rate limiting per endpoint
- Request size limits
- Input validation and sanitization
- SQL injection prevention
- XSS protection

Infrastructure Security:
- VPC isolation
- Security groups configuration
- Network access control lists
- DDoS protection
- Web Application Firewall (WAF)

Audit and Compliance

Audit Logging:

- User action tracking
- API request logging
- System event monitoring
- Compliance event recording

Security Monitoring:
- Intrusion detection system
- Vulnerability scanning
- Security incident response
- Penetration testing

Troubleshooting

Common Issues

Database Connection Issues

Problem: Database connection failures
Symptoms:
- "Database connection timeout" errors
- 500 errors on API requests
- Health check failures

Solutions:
```bash
# Check database status
pg_isready -h localhost -p 5432

# Verify connection string
echo $DATABASE_URL

# Check connection pool settings
# In src/config.py, adjust:
DATABASE_POOL_SIZE = 10
DATABASE_MAX_OVERFLOW = 20
```

Federation Member Communication

Problem: Federation member unreachable
Symptoms:
- Federation verification failures
- Member health check failures
- Trace aggregation timeouts

Solutions:
```python
```

Check member endpoint

import requests
response = requests.get(f"{member_endpoint}/health")
print(response.status_code)

Verify member trust score

trust_score = registry.trust_config.get_trust_score(member_id)
print(f"Trust score: {trust_score}")

Reset member status

await registry.reset_member_status(member_id)
```

## Payment Processing Failures

**Problem:** Payment processor errors
**Symptoms:**
- Payment failures
- Webhook processing errors
- Processor health check failures

**Solutions:**
```bash
## Check processor status

curl -X GET "http://localhost:8000/api/v1/payments/health"

## Verify API keys

echo $STRIPE_API_KEY
echo $PAYPAL_CLIENT_ID

## Check processor configuration

python -c "from src.payments.payment_processor_manager import *; print(create_pa
```

Error Codes

Code	Description	Resolution
E001	Database connection failed	Check database connectivity and credential
E002	Invalid API key	Verify API key is valid and has required permissions
E003	Capsule not found	Verify capsule ID exists in the system
E004	Federation member unreachable	Check member endpoint and network conne
E005	Payment processor error	Verify payment processor configuration and AP
E006	Compliance assessment failed	Review compliance evidence and requireme
E007	Rights validation error	Verify rights chain and ownership
E008	Insufficient permissions	Check user permissions and API key scopes

Debug Mode
```

Enable Debug Logging:
```python
# In src/config.py
LOGGING_LEVEL = "DEBUG"
DEBUG_MODE = True
```

Debug Utilities:
```python
# Debug capsule state
from src.engine.capsule_engine import CapsuleEngine
engine = CapsuleEngine()
await engine.debug_capsule_state(capsule_id)

# Debug federation status
from src.integrations.federated_registry import FederatedModelRegistry
registry = FederatedModelRegistry("debug", "Debug Member")
registry.debug_federation_status()

# Debug payment processing
from src.payments.payment_processor_manager import PaymentProcessorManager
manager = PaymentProcessorManager()
manager.debug_processor_status()
```

Performance Optimization

Database Optimization:
```sql
-- Add indexes for common queries
CREATE INDEX idx_capsules_created_at ON capsules(created_at);
CREATE INDEX idx_capsules_status ON capsules(status);
CREATE INDEX idx_transactions_user_id ON transactions(user_id);
```

Connection Pool Tuning:
```python
# In src/config.py
DATABASE_POOL_SIZE = 20
DATABASE_MAX_OVERFLOW = 40
DATABASE_POOL_TIMEOUT = 30
```

Cache Configuration:
```python
# Redis cache settings
REDIS_URL = "redis://localhost:6379"
```

```
CACHE_TTL = 3600  # 1 hour
CACHE_MAX_CONNECTIONS = 20
```

Conclusion

The UATP Capsule Engine provides a comprehensive platform for AI reasoning trace

For additional support:
- GitHub Issues: https://github.com/your-org/uatp-capsule-engine/issues
- Documentation: https://docs.uatp-capsule-engine.com
- Community Forum: https://forum.uatp-capsule-engine.com
- Email Support: support@uatp-capsule-engine.com

Last Updated: January 2024
Version: 1.0.0